

CẨM NANG THUYẾT TRÌNH DỰ ÁN FASTFOODHUB

Tài liệu chuẩn bị thuyết trình chuyên sâu về Database, Kết nối & Xác thực

Tài liệu này tổng hợp toàn bộ các kiến thức kỹ thuật chi tiết nhất để giúp bạn trả lời trôi chảy các câu hỏi của Hội đồng/Giáo viên về dự án FastFoodHub (ASP.NET Web Forms + C# + MySQL).

PHẦN 1: TÌM HIỂU CHI TIẾT VỀ DATABASE (CƠ SỞ DỮ LIỆU)

Cơ sở dữ liệu của dự án có tên là **fastfooddb**, sử dụng hệ quản trị cơ sở dữ liệu MySQL. Hệ thống được thiết kế theo mô hình quan hệ chuẩn hóa cao để đảm bảo toàn vẹn dữ liệu.

1. Chi tiết các bảng (Tables) trong Database

Hệ thống gồm 7 bảng chính. Dưới đây là cấu trúc chi tiết của từng bảng:

Tên Bảng	Khóa chính	Khóa ngoại	Chức năng của bảng
users	UserId	Không	Lưu trữ thông tin tài khoản của khách hàng và quản trị viên (Username, Password mã hóa, Email, Số điện thoại...).
categories	CategoryId	Không	Phân loại các món ăn (như Burger, Pizza, Đồ uống...) để hiển thị trên thực đơn.
products	ProductId	CategoryId	Lưu trữ thông tin chi tiết món ăn (Tên, Giá, Ảnh, Số lượng tồn kho, Trạng thái Kích hoạt).
carts	CartId	ProductId, UserId	Lưu thông tin giỏ hàng tạm thời của từng người dùng trước khi đặt hàng.
payment	PaymentId	Không	Lưu thông tin giao dịch thanh toán (Tên chủ thẻ, số thẻ, địa chỉ nhận hàng, phương thức thanh toán như COD hay Credit Card).
orders	OrderDetailId	ProductId, UserId, PaymentId	Chi tiết các món ăn có trong một đơn hàng đã đặt, kèm theo trạng thái đơn hàng (Pending, Dispatched, Delivered).
contact	ContactId	Không	Lưu trữ các lời nhắn, góp ý gửi từ biểu mẫu liên hệ ngoài trang chủ.

2. Sơ đồ Quan hệ Thực thể (Entity Relationship Diagram - ERD)

Mối quan hệ giữa các bảng được xây dựng chặt chẽ bằng ràng buộc khóa ngoại để tránh dữ liệu mồ côi (Orphaned Data) khi xóa:

- **Categories → Products (1 : N):** Một danh mục có nhiều sản phẩm. Khi danh mục bị xóa, sản phẩm tương ứng cũng sẽ chịu ràng buộc (có thể bị xóa theo - ON DELETE CASCADE).
- **Users → Carts (1 : N) & Products → Carts (1 : N):** Bảng carts làm cầu nối nhiều-nhiều giữa người dùng và sản phẩm để tạo giỏ hàng dữ liệu tạm thời.
- **Payment → Orders (1 : N):** Một lượt thanh toán có thể chứa nhiều sản phẩm (các dòng chi tiết đơn hàng trong bảng orders sẽ có chung một mã PaymentId).

3. Các Thủ tục Lưu trữ (Stored Procedures - SP) quan trọng

Thay vì viết code SQL trực tiếp trong C#, dự án sử dụng Stored Procedures viết trực tiếp trong MySQL để tối ưu hóa hiệu năng, tăng tính bảo mật (chống SQL Injection) và dễ bảo trì:

- **User_Crud:**

- *Nhiệm vụ:* Xử lý toàn bộ thao tác liên quan đến người dùng.
- *Hành động (p_Action):* SELECT4LOGIN (Xác thực tài khoản khi đăng nhập), SELECT4PROFILE (Lấy thông tin hiển thị ở trang cá nhân), INSERT (Đăng ký tài khoản mới), SELECT4ADMIN (Admin duyệt danh sách người dùng).

- **Cart_Crud:**

- *Nhiệm vụ:* Quản lý giỏ hàng của từng khách hàng.
- *Hành động (Action):* SELECT (Lấy danh sách món ăn trong giỏ để hiển thị), INSERT (Thêm món mới vào giỏ, nếu món đã có thì tự động cộng dồn số lượng), UPDATE (Cập nhật số lượng món khi khách thay đổi trên giao diện), DELETE (Xóa món khỏi giỏ hàng).

- **DashboardSp:**

- *Nhiệm vụ:* Thu thập dữ liệu thống kê cho trang quản trị Admin.
- *Hoạt động:* Thực hiện hàng loạt câu lệnh COUNT trên các bảng users, categories, products, orders và tính SUM doanh thu các đơn hàng có trạng thái *Delivered* để trả về kết quả tổng hợp trong 1 câu truy vấn duy nhất.

- **SellingReport:**

- *Nhiệm vụ:* Lọc và trả về danh sách các đơn hàng đã giao thành công trong khoảng thời gian chỉ định (từ p_FromDate đến p_ToDate) để phục vụ việc lập báo cáo doanh thu của Admin.

TIP: ĐIỂM CỘNG KHI THUYẾT TRÌNH

Hãy nhấn mạnh với giáo viên rằng việc sử dụng Stored Procedures giúp tách biệt tầng dữ liệu (Database) khỏi tầng logic (C#), làm giảm lượng dữ liệu truyền tải qua mạng và bảo mật ứng dụng tốt hơn rất nhiều so với viết SQL thuần (Ad-hoc Query) trong code.

PHẦN 2: CƠ CHẾ KẾT NỐI CƠ SỞ DỮ LIỆU (DATABASE CONNECTION)

Dự án sử dụng cơ chế kết nối truyền thống của .NET Framework là ADO.NET kết hợp với thư viện kết nối MySQL (MySql.Data.MySqlClient).

1. Cấu hình chuỗi kết nối (Connection String)

Chuỗi kết nối được khai báo tập trung trong file **Web.config** để dễ dàng thay đổi thông tin máy chủ mà không cần biên dịch lại code C#:

```
<connectionStrings>
  <add name="cs"
  connectionString="Server=127.0.0.1;Port=3306;Database=fastfooddb;Uid=root;Pwd=MAT_KHAU_CUA_B
  providerName="MySql.Data.MySqlClient" />
</connectionStrings>
```

- **Server=127.0.0.1:** Địa chỉ IP máy chủ database (Localhost).
- **Database=fastfooddb:** Tên cơ sở dữ liệu kết nối.
- **Uid=root:** Tài khoản quản trị tối cao của MySQL.
- **Pwd:** Mật khẩu của tài khoản MySQL.

2. Class Connection.cs và cách quản lý kết nối

Trong code C#, lớp `Connection.cs` chịu trách nhiệm đọc chuỗi kết nối này:

```
public class Connection
{
    public static string GetConnectionString()
    {
        return ConfigurationManager.ConnectionStrings["cs"].ConnectionString;
    }
}
```

Mỗi khi cần làm việc với DB, mã nguồn C# sẽ khởi tạo một đối tượng kết nối `MySqlConnection` và thực thi lệnh qua `MySqlCommand` theo một vòng đời chuẩn khép kín:

```

// 1. Khởi tạo đối tượng kết nối bằng Connection String
using (MySQLConnection con = new MySQLConnection(Connection.GetConnectionString()))
{
    // 2. Định nghĩa Stored Procedure cần gọi
    using (MySQLCommand cmd = new MySQLCommand("DashboardSp", con))
    {
        cmd.CommandType = CommandType.StoredProcedure;

        // 3. Mở kết nối
        con.Open();

        // 4. Thực thi và đọc dữ liệu
        using (MySQLDataReader sdr = cmd.ExecuteReader())
        {
            if (sdr.Read()) {
                // Đọc kết quả xử lý dữ liệu...
            }
        } // Tự động đóng đầu đọc sdr
    } // Tự động giải phóng tài nguyên của đối tượng cmd
} // Khởi using tự động đóng kết nối (con.Close()) và giải phóng tài nguyên RAM, tránh rò rỉ kết nối (Connection Leak)

```

PHẦN 3: CƠ CHẾ XÁC THỰC VÀ PHÂN QUYỀN (AUTHENTICATION & AUTHORIZATION)

Dự án quản lý phiên làm việc của người dùng bằng **Session State** tích hợp sẵn của ASP.NET.

1. Quy trình Đăng ký tài khoản (Registration.aspx)

1. Người dùng điền đầy đủ các thông tin cá nhân trên giao diện biểu mẫu đăng ký.
2. Code C# phía Backend thực hiện gọi Stored Procedure `User_Crud` với tham số hành động `p_Action = "INSERT"` để đẩy dữ liệu người dùng mới vào bảng `users`. Mật khẩu được xử lý lưu trữ an toàn.

2. Quy trình Đăng nhập & Tạo phiên làm việc (Login.aspx.cs)

Khi người dùng điền thông tin và bấm nút Đăng nhập, Backend sẽ khởi tạo lệnh kiểm tra:

```

con = new MySqlConnection(Connection.GetConnectionString());
cmd = new MySqlCommand("User_Crud", con);
cmd.CommandType = CommandType.StoredProcedure;
cmd.Parameters.AddWithValue("p_Action", "SELECT4LOGIN");
cmd.Parameters.AddWithValue("p_Username", txtUsername.Text.Trim());
cmd.Parameters.AddWithValue("p_Password", txtPassword.Text.Trim()); // Mật khẩu người
dùng nhập
// ... các tham số khác gửi chuỗi rỗng

```

Sau khi chạy truy vấn, hệ thống kiểm tra số dòng dữ liệu trả về:

- **Nếu khớp thông tin tài khoản (Số dòng = 1):** Hệ thống tiến hành Phân quyền dựa trên tên đăng nhập:
 - **Nếu tài khoản đăng nhập là "admin":**

Thiết lập: `Session["admin"] = dbUsername;`

Điều hướng thẳng đến trang quản trị: `Response.Redirect("../Admin/Dashboard.aspx");`
 - **Nếu tài khoản là các khách hàng thông thường:**

Thiết lập: `Session["username"] = dbUsername;` và `Session["userId"] = dt.Rows[0]["UserId"];`

Điều hướng về trang chủ mua sắm: `Response.Redirect("Default.aspx");`
- **Nếu không tìm thấy dòng dữ liệu nào:** Hệ thống hiển thị thông báo lỗi tài khoản hoặc mật khẩu không chính xác.

3. Phân quyền và Bảo vệ trang (Authorization)

Để ngăn chặn việc người dùng thường hoặc khách vãng lai cố tình gõ trực tiếp URL để xâm nhập trái phép vào các trang quản trị (ví dụ gõ `/Admin/Dashboard.aspx`), hệ thống sử dụng cơ chế kiểm tra Session chặt chẽ tại sự kiện khởi tải trang (`Page_Load`):

Tại các trang của Admin (Ví dụ `Dashboard.aspx.cs`):

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)
    {
        // Kiểm tra xem phiên làm việc của Admin có tồn tại hay không
        if (Session["admin"] == null)
        {
            // Nếu không phải Admin, lập tức đá người dùng quay lại trang đăng nhập
            Response.Redirect("../User/Login.aspx");
        }
        else
        {
            // Nếu đúng là Admin thì mới thực hiện tải dữ liệu Dashboard
            DashboardCount dashboard = new DashboardCount();
            Session["category"] = dashboard.Count("CATEGORY");
            // ... tiếp tục tải các thông số khác
        }
    }
}
```

Kiểm soát hiển thị động trên thanh điều hướng (User.Master.cs): Thanh Menu đầu trang sẽ tự động thay đổi nút bấm dựa trên trạng thái đăng nhập thực tế:

```
if (Session["userId"] != null)
{
    lbLoginOrLogout.Text = "Logout"; // Hiển thị tùy chọn Đăng xuất
    Utils utils = new Utils();
    // Lấy số lượng món có trong giỏ hàng thực tế để cập nhật số hiển thị Badge thông báo
    Session["cartCount"] =
        utils.cartCount(Convert.ToInt32(Session["userId"])).ToString();
}
else
{
    lbLoginOrLogout.Text = "Login"; // Người dùng chưa đăng nhập, hiển thị nút Đăng nhập
    Session["cartCount"] = "0";
}
```

4. Quy trình Đăng xuất (Logout)

Khi người dùng hoặc Admin nhấn chọn nút Logout, mã nguồn xử lý sự kiện phía Backend sẽ gọi hàm:

```
Session.Abandon(); // Hủy bỏ hoàn toàn phiên làm việc hiện tại, xóa toàn bộ dữ liệu lưu
trong Session
Response.Redirect("Login.aspx"); // Đưa người dùng an toàn quay lại trang đăng nhập ban
đầu
```

Việc sử dụng lệnh `Session.Abandon()` đảm bảo mọi dữ liệu phiên cũ bị xóa sạch hoàn toàn khỏi bộ nhớ RAM của hệ thống máy chủ, ngăn chặn rủi ro chiếm đoạt phiên và bảo vệ an toàn tuyệt đối cho người dùng tiếp theo sử dụng trên cùng một trình duyệt.